

Appendix C

Program Libraries

*Since the programming is likely to be the
main bottleneck in the use of an electronic computer
we have given a good deal of thought to the
preparation of standard routines of considerable generality for the
more important processes involved in computation.*

*By this means we hope to reduce the time
taken to code up large-scale computing problems,
by building them up, as it were,
from prefabricated units.*

— J. H. WILKINSON, *The Automatic Computing Engine at the
National Physical Laboratory* (1948)

*In spite of the self-contained nature of the linear algebra field,
experience has shown that even here
the preparation of a fully tested set of algorithms
is a far greater task than had been anticipated.*

— J. H. WILKINSON and C. REINSCH, *Handbook for
Automatic Computation: Linear Algebra* (1971)

In this appendix we briefly describe some of the freely available program libraries that have been mentioned in this book. These packages are all available from netlib (see §B.2).

C.1. Basic Linear Algebra Subprograms

The Basic Linear Algebra Subprograms (BLAS) are sets of Fortran primitives for matrix and vector operations. They cover all the common operations in linear algebra. There are three levels, corresponding to the types of object operated upon. In the examples below, x and y are vectors, A, B, C are rectangular matrices, and T is a square triangular matrix. Names of BLAS routines are given in parentheses. The leading “x” denotes the Fortran data type, whose possible values are some or all of

```
S  real
D  double precision
C  complex
Z  complex*16, or double complex
```

Level 1: [774, 1979] *Vector operations*. Inner product: $x^T y$ (xdot); $y \leftarrow \alpha x + y$ (xAXPY); vector 2-norm $(y^T y)^{1/2}$ (xNRM2); swap vectors $x \leftrightarrow y$ (xSWAP); scale a vector $x \leftarrow \alpha x$ (xSCAL); and other operations.

Level 2: [347, 1988], [348, 1988] *Matrix–vector operations*. Matrix times vector (gaxpy): $y \leftarrow \alpha Ax + \beta y$ (xGEMV); rank-1 update: $A \leftarrow A + \alpha xy^T$ (xGER); triangular solve: $x \leftarrow T^{-1}x$ (xTRSV); and variations on these.

Level 3: [342, 1990], [343, 1990] *Matrix–matrix operations*. Matrix multiplication: $C \leftarrow \alpha AB + \beta C$ (xGEMM); multiple right-hand side triangular solve: $A \leftarrow \alpha T^{-1}A$ (xTRSM); rank- r and rank- $2r$ updates of a symmetric matrix (xSYRK, xSYR2K); and variations on these.

The BLAS are intended to be called in innermost loops of linear algebra codes. Usually, most of the computation in a code that uses BLAS calls is done inside these calls. LINPACK [341, 1979] uses the level-1 BLAS throughout (model Fortran implementations of the level-1 BLAS are listed in [341, 1979, App. D]). LAPACK [20, 1999] exploits all three levels, using the highest possible level at all times.

Each set of BLAS comprises a set of *subprogram specifications* only. The specifications define the parameters to each routine and state what the routine must do, but not how it must do it. Thus the implementor has freedom over the precise implementation details (loop orderings, block algorithms, special code for special cases) and even the method (fast versus conventional matrix multiply), but the implementation is required to be numerically stable, and code that tests the numerical stability is provided with the model implementations [343, 1990], [348, 1988].

For more details on the BLAS and the advantages of using them, see the defining papers listed above, or, for example, [349, 1998] or [509, 1996, Chap. 1].

Automated techniques for producing BLAS tuned to particular machines have been developed in the ATLAS project [1218, 2001].

More recently, a BLAS Technical Forum Standard has been produced that extends the functionality of the original level-1, -2, and -3 BLAS, adding BLAS to handle sparse and banded matrices, further dense matrix computations (many based on LAPACK auxiliary routines), and extended and mixed precision computations [88, 2002], [137, 2001]. It provides bindings for Fortran 95, Fortran 77, and C. For more on the Extended and Mixed Precision BLAS see §27.10. The Web site of the BLAS Technical Forum is <http://www.netlib.org/blas/blast-forum/>

C.2. EISPACK

EISPACK is a collection of Fortran 66 subroutines for computing eigenvalues and eigenvectors of matrices [1047, 1976], [455, 1977]. It contains 58 subroutines and 13 drivers. The subroutines are the basic building blocks for eigensystem computations; they perform such tasks as reduction to Hessenberg form, computation of some or all of the eigenvalues/vectors, and back transformations, for various types of matrix (real, complex, symmetric, banded, etc.). The driver subroutines provide easy access to many of EISPACK's capabilities; they call from one to five other EISPACK subroutines to do the computations.

EISPACK is primarily based on Algol 60 procedures developed in the 1960s by 19 different authors and published in the journal *Numerische Mathematik*. An edited collection of these papers was published in the *Handbook for Automatic Computation* [1246, 1971].

C.3. LINPACK

LINPACK is a collection of Fortran 66 subroutines that analyse and solve linear equations and linear least squares problems [341, 1979]. The package solves linear systems whose matrices are general, banded, symmetric indefinite, symmetric positive definite, triangular, or tridiagonal. In addition, the package computes the QR and singular value decompositions and applies them to least squares problems. All the LINPACK routines use calls to the level-1 BLAS in the innermost loops; thus most of the floating point arithmetic in LINPACK is done within the level-1 BLAS.

C.4. LAPACK

LAPACK [20, 1999] was released on February 29, 1992. As the announcement stated, "LAPACK is a transportable library of Fortran 77 subroutines for solving the most common problems in numerical linear algebra: systems of linear equations, linear least squares problems, eigenvalue problems, and singular value problems. It has been designed to be efficient on a wide range of modern high-performance computers."

LAPACK has been developed since 1987 by a worldwide team of numerical analysts. It can be regarded as a successor to LINPACK and EISPACK, as it

has virtually all their capabilities and much more besides. LAPACK improves on LINPACK and EISPACK in four main respects: speed, accuracy, robustness, and functionality. It was designed at the outset to exploit the level-3 BLAS.

Interfaces from LAPACK (or translations of LAPACK) to other languages are available for Fortran 95 [65, 2001], C, C++, and Java; see the links on the LAPACK home page at <http://www.netlib.org/lapack/>

A related project ScaLAPACK has produced a subset of LAPACK routines redesigned for distributed memory parallel machines [135, 1997].

Other work includes developing codes that take advantage of the careful rounding and exception handling of IEEE arithmetic [331, 1994]. For more details of all these topics see [20, 1999].

LAPACK undergoes regular updates, which are announced on the electronic newsletter NA-Digest. At the time of writing, the current release is version 3.0, last updated May 31, 2000, and the package contains over 1000 routines and over 735,000 lines of Fortran 77 code, including testing and timing code.

Mark 16 onward of the NAG Fortran 77 Library contains much of LAPACK in Chapters F07 and F08.

C.4.1. Structure of LAPACK

The LAPACK routines can be divided into three classes.

The *drivers* solve a complete problem. The *simple* drivers have a minimal specification, while the *expert* drivers have additional capabilities of interest to the sophisticated user. The *computational* routines perform individual tasks such as computing a factorization or reducing a matrix to condensed form; they are called by the drivers. The *auxiliary* routines perform relatively low-level operations such as unblocked factorizations, estimating or computing matrix norms, and solving a triangular system with scaling to prevent overflow.

The driver and computational routines have names of the form **xyyzzz**. The first letter specifies the data type, which is one of **S**, **D**, **C**, and **Z**. The second two letters refer to the type of matrix. A partial list of types is as follows (there are 27 types in all):

BD	bidiagonal
GB	general band
GE	general
GT	general tridiagonal
HS	upper Hessenberg
OR	(real) orthogonal
PO	symmetric or Hermitian positive definite
PT	symmetric or Hermitian positive definite tridiagonal
SB	(real) symmetric band
ST	(real) symmetric tridiagonal
SY	symmetric
TR	triangular (or quasi-triangular)

The last three characters specify the computation performed.

TRF	factorize
TRS	solve a (multiple right-hand side) linear system using the factorization
CON	estimate $1/\kappa_1(A)$ (or compute it exactly when A is tridiagonal and symmetric positive definite or Hermitian positive definite)
RFS	apply fixed precision iterative refinement and compute the componentwise relative backward error and a forward error bound
TRI	use the factorization to compute A^{-1}
EQU	compute factors to equilibrate the matrix

The auxiliary routines follow a similar naming convention, with most of them having `yy = LA`.